Constraint Satisfaction Problems

High-Yield Master Revision Document

Purpose of This Document

This document is designed for students who want a strong, efficient, exam-focused revision source for Constraint Satisfaction Problems (CSPs). It is not written as a full lecture rewrite. Instead, it extracts the most important ideas, distinctions, algorithms, and traps that students are most likely to need in tests, quizzes, short questions, long questions, and final revision.

> **Main Goal:**
>
> If a student studies this document properly, they should be able to explain what a CSP is, model basic problems, compare the major algorithms and heuristics, understand why propagation matters, and distinguish between the major solution strategies.

1. Core Idea of a CSP

A Constraint Satisfaction Problem is a problem in which:

- there is a set of **variables**,
- each variable has a **domain** of possible values,
- there is a set of **constraints** restricting which combinations of values are allowed.

The goal is to find a **complete and consistent assignment**.

> **Master Definition:**
>
> A CSP asks us to assign values to variables from their domains so that all constraints are satisfied.

Two Running Examples

Element	Map Coloring	Exam Scheduling
Variables	Regions	Exams
Domains	Colors	Time slots
Constraints	Neighbors must have different colors	Conflicting exams must be at different times
Goal	Valid coloring	Valid schedule

Things to Note:

- A solution is not just any assignment. It must be both complete and consistent.
- CSPs are different from ordinary search problems because the main focus is satisfying constraints, not following action sequences.
- Good modeling is the foundation of good solving.

2. Modeling a Problem as a CSP

Whenever you are given a problem, think in this order:

1. What are the variables?
2. What values can each variable take?
3. What rules limit legal combinations?

Modeling Checklist

- Variables should represent the things that need values.
- Domains should include all legitimate choices.
- Constraints should clearly express the rules of the problem.

Common Trap:

Students often confuse variables with values. For example, in exam scheduling, the exams are the variables, not the time slots. The time slots are the domain values.

Constraint Types

- **Unary constraint:** restricts one variable.
- **Binary constraint:** relates two variables.
- **Higher-order constraint:** involves more than two variables.

Constraint Type	Map Coloring Example	Exam Scheduling Example
Unary	Less central in the standard example	`ENG101 ≠ Mon-AM`
Binary	`WA ≠ NT`	`CS101 ≠ CS102`
Higher-order	A rule involving several regions at once	A rule involving several exams at once

3. Why Naive Search Is Wasteful

In a normal search problem, different action orders can produce meaningfully different states. In a CSP, assigning the same set of variable-value pairs in different orders usually leads to the same logical partial assignment.

This means naive search wastes effort by exploring redundant paths.

> **High-Yield Point:**
>
> The key reason naive search is inefficient for CSPs is that assignment order is often irrelevant, but naive search still treats it as important.

4. Backtracking Search

Backtracking is the standard exact search method for CSPs.

It works by:

1. choosing an unassigned variable,
2. trying a value from its domain,
3. checking whether the assignment is consistent,
4. continuing if consistent,
5. backtracking if inconsistency is found.

Why Backtracking Matters

- It uses partial assignments rather than complete blind trial.
- It detects failure as soon as a violation appears.
- It is systematic and complete.

> **Master Line:**
>
> Backtracking improves CSP solving by assigning one variable at a time and abandoning partial assignments as soon as they violate constraints.

Backtracking in the Two Examples

- **Map coloring:** assign one region a color and continue only if no adjacent region rule is violated.
- **Exam scheduling:** assign one exam a time slot and continue only if no clash or unary restriction is violated.

> **Common Trap:**
>
> Backtracking is not a different problem formulation. It is a solution method for the same CSP.

5. Heuristics That Improve Backtracking

Basic backtracking is correct but may still be slow. Heuristics improve efficiency by making smarter choices.

MRV: Minimum Remaining Values

Choose the variable with the fewest legal values left.

- Main idea: tackle the most constrained variable first.
- Main benefit: failure is discovered earlier.

Degree Heuristic

If there is a tie in MRV, choose the variable that constrains the largest number of other unassigned variables.

- Main idea: choose the variable with greatest future impact.

LCV: Least-Constraining Value

Choose the value that rules out the fewest choices for neighboring variables.

- Main idea: preserve future flexibility.

Heuristic	What It Chooses	Main Goal
MRV	Variable	Expose failure early
Degree	Variable	Maximize constraint impact
LCV	Value	Preserve future options

Most Important Distinction:

MRV and Degree are variable-ordering heuristics. LCV is a value-ordering heuristic.

Common Trap:

Students often mix up MRV and LCV. MRV chooses which variable to assign next. LCV chooses which value to try for a chosen variable.

6. Forward Checking

Forward checking is a look-ahead method used during search.

After assigning a variable, it removes inconsistent values from the domains of neighboring unassigned variables.

Why It Helps

- It detects some failures earlier than plain backtracking.
- It reduces wasted exploration.
- It gives domain information that helps MRV work better.

Example Pattern

- **Map coloring:** if `WA = red`, then red is removed from adjacent regions.
- **Exam scheduling:** if `CS101 = Mon-AM`, that slot is removed from conflicting exams.

> **Key Idea:**
>
> Forward checking does not just react to current inconsistency. It anticipates immediate future inconsistency.

Main Limitation

Forward checking only looks one step ahead. It does not fully propagate all indirect consequences through the constraint network.

7. Constraint Propagation and Arc Consistency

Constraint propagation is the general idea of repeatedly using constraints to remove impossible values from domains.

Arc consistency is one of the most important forms of propagation for binary CSPs.

Arc Consistency

An arc from X to Y is consistent if for every value in X's domain, there is some compatible value in Y's domain.

If a value in X has no supporting value in Y, it must be removed.

AC-3

AC-3 is the standard algorithm used to enforce arc consistency by repeatedly checking arcs and propagating reductions.

Method	What It Does	Strength	Limitation
Forward Checking	Prunes immediate neighbors after assignment	Simple and useful	Limited propagation
Arc Consistency / AC-3	Propagates support-based pruning through arcs	Stronger pruning	Still does not guarantee a full solution

> **Most Important Distinction:**

> Forward checking sees immediate consequences of a new assignment. Arc consistency pushes pruning more deeply through the network.

> **Common Trap:**
>
> Do not assume that if a CSP is arc-consistent, it must already be solved. Local consistency does not guarantee a global solution.

8. Problem Structure Matters

Some CSPs are hard not because of the number of variables alone, but because of how the variables are connected.

Independent Subproblems

If the constraint graph has disconnected components, each component can be solved separately.

Tree-Structured CSPs

If the constraint graph has no cycles, solving becomes much easier.

- Trees are easier because dependencies do not loop back.
- Consistency and assignment can be handled in a more controlled direction.

Cutset Conditioning

If a small set of variables can be removed to make the graph a tree, try assignments to that set first, then solve the remaining tree efficiently.

Structure Idea	Main Advantage
Independent subproblems	Solve components separately
Tree structure	No cycles, easier solving
Cutset conditioning	Break cycles by fixing a small set first

> **Exam-Friendly Line:**
>
> The structure of a CSP can be as important as the algorithm used to solve it.

9. Local Search and Min-Conflicts

Local search approaches CSPs differently from backtracking.

- Backtracking builds a solution step by step from partial assignments.
- Local search starts with a complete assignment and tries to repair conflicts.

Min-Conflicts

Choose a variable involved in a conflict, then assign it the value that minimizes the number of violated constraints.

Why It Can Work Well

- Many large CSPs have solutions close to nearly-correct assignments.
- Repairing a few bad choices can be faster than building from scratch.

Main Limitation

Local search can get stuck in local minima or plateaus, so it may need randomness or restarts.

> **Most Important Distinction:**
>
> Backtracking works with partial assignments. Local search works with complete but imperfect assignments.

10. Side-by-Side Master Comparisons

Backtracking vs Local Search

Feature	Backtracking	Local Search
Assignment type	Partial	Complete
Approach	Construct carefully	Repair conflicts
Strength	Systematic and complete	Often effective on large problems
Weakness	Can be slow	Can get stuck

MRV vs Degree vs LCV

Heuristic	Type	Question It Answers
MRV	Variable ordering	Which variable is closest to failure?
Degree	Variable ordering	Which variable constrains others the most?
LCV	Value ordering	Which value preserves the most future flexibility?

Forward Checking vs Arc Consistency

Feature	Forward Checking	Arc Consistency

Depth of pruning	Immediate neighbors	More global propagation through arcs
When used	After assignment	As a stronger propagation process
Main strength	Simple early failure detection	Stronger domain reduction

11. What Students Most Commonly Get Wrong

- Confusing variables with domain values.
- Thinking heuristics change correctness rather than efficiency.
- Mixing up MRV and LCV.
- Thinking forward checking and arc consistency are the same thing.
- Assuming arc consistency guarantees a solution.
- Forgetting that backtracking and local search use very different philosophies.
- Ignoring problem structure.

High-Risk Misunderstanding:

Heuristics and propagation do not replace backtracking in the standard exact approach. They strengthen it by guiding choices and pruning impossible values.

12. Most Likely Written Exam Angles

Students should be ready to answer questions like:

- Define a CSP with an example.
- Differentiate naive search and backtracking for CSPs.
- Explain MRV, Degree heuristic, and LCV.
- Explain forward checking and its limitations.
- Explain arc consistency and AC-3.
- Compare forward checking and arc consistency.
- Explain why problem structure matters in CSP solving.
- Explain min-conflicts and how it differs from backtracking.
- Model a scheduling or coloring problem as a CSP.

Revision Strategy:

If a student can answer those nine question types clearly, they are covering most of the high-yield core of the chapter.

13. Master Revision Sheet

Topic	One-Line Revision
CSP	Assign values to variables so that all constraints are satisfied.
Variables	The entities that need values.
Domains	The possible values each variable may take.
Constraints	The rules limiting legal combinations of values.
Backtracking	Systematic search over partial assignments.
MRV	Pick the variable with the fewest legal values left.
Degree heuristic	Break ties using the variable that constrains the most others.
LCV	Pick the value that removes the fewest options for neighbors.
Forward checking	Prune neighbor domains immediately after assignment.
Arc consistency	Remove values that have no support in neighboring domains.
AC-3	Algorithm that enforces arc consistency through repeated checking.
Independent subproblems	Disconnected graph components can be solved separately.
Tree-structured CSP	No cycles means easier solving.
Cutset conditioning	Fix a small set of variables to turn the rest into a tree.
Local search	Start with a complete assignment and repair conflicts.
Min-conflicts	Change a conflicted variable to reduce total violations.

14. Final Last-Minute Revision Paragraph

A Constraint Satisfaction Problem consists of variables, domains, and constraints. The objective is to produce a complete assignment that satisfies all constraints. CSPs are usually solved exactly with backtracking, whose efficiency can be improved using MRV, Degree heuristic, and LCV. Forward checking and arc consistency reduce wasted search by pruning impossible values early. The structure of the constraint graph matters greatly: disconnected components, trees, and small cutsets can make solving easier. Local search, especially min-conflicts, offers a different strategy by starting from a complete assignment and repairing conflicts.

> **Final Master Line to Memorize:**
>
> CSP solving is about correct modeling, intelligent search, smart pruning, structural advantage, and conflict repair.

End of High-Yield Master Revision Document